

Automatic Manga Colorization

Giulio Arecco
Politecnico di Torino

Alberto Cagnazzo
Politecnico di Torino

Abstract

The problem of image colorization has been studied and analyzed broadly in the past years, especially in the field of automatic colorization using deep neural networks. In this work we want to explore the possibility of using state-of-art architectures, initially designed for coloring real-life images, on the task of automatic manga drawing colorization, in order to understand if CNNs (non-generative and generative) are well suited for this specific task.

1. Introduction

Analyzing the grayscale version of a drawing and inferring the appropriate colors for each region is a highly complex task. Unlike real-world image colorization, where natural elements and common objects (e.g., grass, dirt, roads) often adhere to predictable color patterns, drawings, especially manga, are not constrained by the colors of the real world. Artists have the creative freedom to use any color palette, making it nearly impossible to determine the intended colors of a handmade environment or character without additional context.

In real-world image colorization, the problem is relatively linear due to the inherent consistency in natural gradients and object colors, although exceptions exist. In contrast, the colorization of drawings is considered an ill-posed problem because there is no single correct answer. The absence of predefined color rules in artistic works introduces significant uncertainty, especially when dealing with fantastical or highly stylized elements.

To address this challenge, we propose a different approach: training a deep neural network specifically on a single manga. By focusing on a specific work, the model can learn the unique color schemes, character designs, and environmental elements envisioned by the original author. This targeted training allows the model to recognize and colorize characters, objects, and scenarios accurately, preserving the artistic integrity of the manga while automating the colorization process.

2. Dataset

In image colorization, having a substantial amount of data is essential for the network to extract both spatial and color information effectively. This is especially important because our approach relies solely on the L channel of the image for inference, without incorporating additional inputs, as seen in other drawing colorization architectures.

The primary challenge in creating a dataset for this type of task lies in obtaining colored drawings that are consistent in style, color schemes, and other elements such as text. Considering our objective is to apply colorization to a specific manga, we selected one with a large number of pages that has also been (almost) fully colorized. Specifically, we used a fan-made color version of "One Piece", which provides approximately 15,000 images, including outliers and text-only pages. We will only consider a subset of those images because our computational resources are limited and so it would be impossible for us to train locally on a dataset this big. Additionally, we used images from ¹, a database of royalty-free anime drawings accessible via an API. However, this dataset presents a significant challenge: it contains millions of images with diverse styles, shapes, proportions, and color schemes, as they are created by various artists.

Randomly selecting drawings from such a dataset and feeding them to the network would lead to suboptimal results. Instead, we refined our dataset by incorporating images tagged as "one.piece" to ensure visual coherence in the data distribution. This approach is intended to provide the network with drawings that maintain a consistent color palette for the same characters, which should theoretically lead to better results.

However, this strategy does not guarantee improvements due to another issue in the data distribution: the variability in artistic styles. Since the drawings are fan-made, each artist's style introduces differences, even when depicting the same character (Fig. 1). Additionally, anyone can upload images under a specific tag, so even if explicit filtering is applied to a given manga, not all drawings will necessarily be valid. This stylistic diversity presents an extra challenge

¹safebooru.org

in achieving consistent colorization results. For this reason we will try to train the network on both datasets independently, evaluating its performance and understanding how it works with certain kind of data. The objective is to allow the network to learn the structure of the manga tables (evaluating and understanding the text, dialogue box, panels, ecc.) and, at the same time, provide it with more information about characters’ color palette and shading style, even with the limitations discussed previously.

2.1. Data Pre-Processing

The entire model works on normalized LAB channels: the model receives as input the Lightness component of the image and produces a the AB channels, that are then de-normalized and used to build the output of the network. The model itself is not quite *end-to-end*, but *pre-* and *post-* processing functions can also be added to the forward pass, allowing a “simulated” end-to-end usage. We have defined a specific class, *ImageNorm*, for referencing all static processing functions, such as *image-resizing*, *tensor converting* and *post-processing*. We followed the implementation of Zhang et al.[6] in order to reproduce the same behavior, using *z_score* normalization over our own data distribution *mean* and *standard deviation*. We will further elaborate on the rationale behind our choice of Zhang et al.’s model in section 3.2.

- The normalization is carried out by the following relations in order to map possible values of the L channel and AB channels to a smaller, *zero-mean* range:

$$L_{normalized} = \frac{L_{channel} - \lambda}{std}$$

$$AB_{normalized} = \frac{AB_{channel} - \lambda}{std}$$

The *std* and *mean* computed over our dataset will be listed later.

2.2. Dataset Subdivision and Training/Test approach

Given that our two reference datasets, consisting of approximately 6000 images, are composed of very sparse data and, even if inspired by the same work, present substantial differences, we trained two different models, one for each dataset, subdividing the data using these proportions:

- Around 500 images were taken randomly to compose the *test set*
- The remainder was split with a ratio of 80/20% between *training set* and *validation set*

The development of the final architectures began with a series of tests on a small batch of data, along with some

training on the entire dataset. Specifically, we attempted to overfit the data to identify optimal hyperparameter values and input processing choices. Additionally, this approach allowed us to quickly assess whether the network was performing the task correctly and determine if architectural adjustments were necessary. We will discuss the hyperparameter values in *Section 3.6* and the concepts and inner-knowledge that we obtained during this phase in *Section 4*.

3. Methods

The problem of image colorization has been tackled from many different perspectives and using a broad variety of models and architectures, each of them trying to solve the task with a completely different approach.

Let us now see which are the most valid networks for both real-world and drawn images colorization.

3.1. Generative Networks Approach

The “natural” way of reaching our goal is, of course, employing a **Generative Adversarial Network**, mainly because colorization is a specific case of style-transferring, a problem often solved with GANs.

The first network that effectively found a solution not only in the task of image colorization but, in general, in any type of *learnable image-to-image translation* is the Pix2Pix architecture: a GAN model that proposes a solution to a series of tasks, such as photo synthetization, object reconstruction, style transfer, and, also, image colorization. The model itself is composed of a U-Net that plays the role of the generator and a Patch GAN network that manages the discrimination. This architecture addresses these challenges by adopting a fundamental principle: the network must serve as a “general-purpose solution that applies the same generic approach to problems that traditionally would require very different loss formulations” [2]. Later in our solution, we will draw inspiration from the Pix2Pix framework to define the adversarial component of our architecture.

A more complex solution is proposed by the work of Shimizu et al[4], that tries to combine the classical adversarial approach with a novel architecture. This model does not tackle specifically automatic colorization but a semi-automatic one, in which two inputs are required for the network inference: Screen Tones (bw drawing) + Flat Colored images. From these two inputs, the network learns how to fully colorize the drawings by mixing shadows and intensities defined by screen tones with colors specified by the second input. As we will discuss later in *Section 4.5*, the problem of drawing colorization is rarely addressed directly using only the grayscale image as input. Consequently, state-of-the-art models that achieve impressive results on this task



Figure 1. Comparison between different types of images of the same character from *safebooru*

typically rely on complex architectures that incorporate fusion levels and multi-stream inputs.

3.2. Our Approach

The purpose of our work is to try to use an **existing** image colorization model, different from the ones listed above, that is based on a completely different method to resolve the problem. We used the Zhang et al [6] colorization network, that was initially intended for real-world images colorization and, in the original work, trained over the ImageNet dataset. The Zhang network uses a convolutional approach that extracts a probability distribution over quantized AB values for each pixel and then outputs the AB channel for each pixel in the input image. We will take this architecture and try to adapt it to our objectives by first training it on our datasets, and then adding an adversarial component to the loss, effectively building a GAN network by using the Zhang network as the Generator and two types of discriminators for the adversarial component of the loss. We have chosen this approach because it takes our main objective (which was using the Zhang network for coloring drawings) and extends it using a generative approach. The other main reason is that, differently from what the Zhang model does, our model cannot learn common color distributions and, most importantly, cannot exploit *class rebalancing* [6] to resolve the problem of desaturated color images. The network, in fact, tends to be biased towards low ab values due to the background and text color (that, in the case of manga tables, occupy the majority of the image surface). To solve this issue and try to obtain more vivid colors, we need to introduce a generative component to the architecture that enhances the resulting AB channel generated by the network.

3.2.1 First Model Proposal

As already suggested, our model is based on a classical GAN that uses a Zhang network as the generator: The generator takes as input the black and white version (more precisely, the L channel) of the image and applies a series of convolutional layers, each of them followed by batch normalization. The out of all the final convolutional layer is a tensor that contains the probability distribution $\hat{Z}$ over AB

quantized bins, that is a **discrete probability distribution** over the possible output colors for the input image. The model uses this predicted $\hat{Z}$ and compares it to the original color distribution Z to learn how to effectively distribute colors in the predicted *ab* channel. Finally, the last layer is a transpose convolution that maps the $\hat{Z}$ to $\hat{Y} = AB$ channels of the final image.

3.2.2 Discriminator

On the other hand, we experimented with two different discriminators in our architecture: one based on a deep CNN and the other derived from the *pix2pix* architecture. Both networks output a binary classification, distinguishing between real and fake samples, as is standard for discriminator networks in GANs.

In the first model proposal, the discriminator has all the "classical" characteristics of a CNN discrimination network: we used *LeakyRELU* as the activation function and applied *Batch Normalization* after each convolutional layer but the very last one, which produces a one-dimensional output. The network architecture is described in fig.3

3.2.3 Second Model Proposal

The second model that we trained uses a *PatchGAN* discriminator, a convolutional network that works on overlapping sub-patches of the original image, giving a classification result for each of them [2]. As we can observe by the structure of the network, the main difference from the previous discriminator is that this architecture produces a matrix as output, with a binary classification for each sub-patch of the input image. Another key difference is that this network takes not only the generated (or real) colored image, but also the associated L channel.

3.3. Weight Initialization

Proper weight initialization is crucial for stabilizing the training process and improving the convergence of deep neural networks. In our trained models, we used different initialization strategies for the generator and discriminator to optimize their respective performance.

For the generator, we employed **He initialization** (also

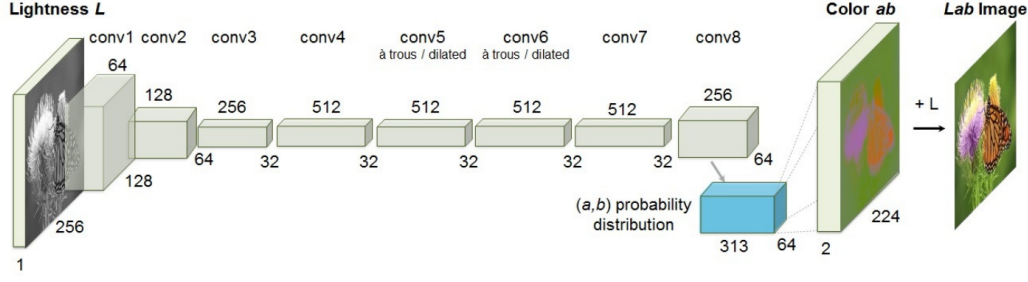


Figure 2. Structure of the Zhang Colorization model [6]

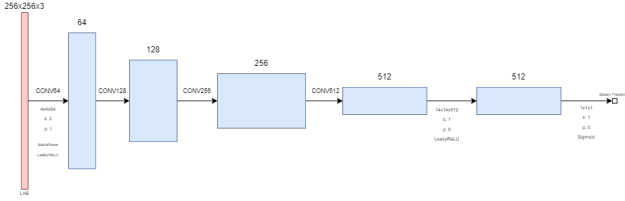


Figure 3. First discriminator model. Takes the LAB image as input for discrimination

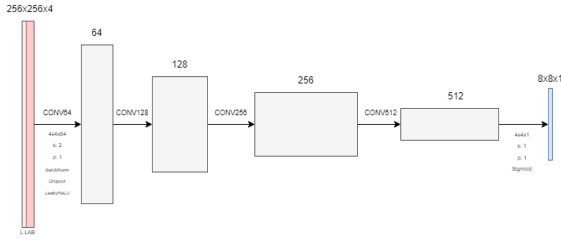


Figure 4. PatchGAN discriminator model. Takes the L channel concatenated to the input image for discrimination

known as *Kaiming* initialization), which is particularly effective for networks using *ReLU* or *LeakyReLU* activation functions. This type of initialization ensures that the variance of activations remains stable throughout the network, preventing vanishing or exploding gradients. It is based on a *Zero-mean Gaussian Distribution*, with:

$$W \sim N(0, \frac{2}{n_{in}})$$

$$Mean = 0, Variance = \frac{2}{n_{in}}$$

For the discriminator, we used a custom weight initialization scheme that resembles the **DCGAN-style initialization** [3], where:

- Convolutional layer weights are initialized with a *normal distribution* (mean = 0, standard deviation = 0.02).
- Batch normalization layer weights are also drawn from

a normal distribution (mean = 1, standard deviation = 0.02), while biases are set to 0.

This initialization helps maintain stable gradient flow and prevents mode collapse in adversarial training. By using these techniques, the model should achieve more reliable training dynamics and stability.

3.4. Loss Functions

The loss functions used in our model, in their different versions, are based on a series of hand-made and preexistent functions, both for the generator and for the discriminator.

3.4.1 Multinomial Cross Entropy Loss

Loss derived from the one proposed by the paper [6], specifically formulated to work on the quantized AB distribution of both the ground truth and predicted AB values. As stated in the cited paper, using an L_2 Euclidean loss would be reductive and not robust, because of the "multimodal nature of the colorization problem" [6]. Hence, to **avoid desaturation** and "sepia tone", given by the averaging approach of the L_2 loss, we will compute the loss over the mapped AB values in $Zspace$: this mapping is based on a function that describes the **probability distribution over possible quantized ab values** (in this case 313 bins) for each pixel in the image. Specifically, this operation is done for a distribution with $H = W = 64$ in one of the *last layers* of the network. The resulting distribution will then be mapped to the output image (size of 256) by the *transpose convolution layer*.

The **loss function** itself is defined as:

$$L_{cl}(\hat{Z}, Z) = - \sum_{h,w} Z_{h,w} \sum_q Z_{h,w,q} \log(\hat{Z}_{h,w,q})$$

$$\text{Where } Z \in [0, 1]^{H \times W \times Q}$$

3.4.2 Inverse H Mapping

Starting from the principle that the last layer of the network performs an operation that maps $\hat{Z}$ to $\hat{Y} = AB$ color

distribution, we need a function that performs the inverse operation, starting from the AB channels of ground truth and producing the predicted Z distribution. Specifically, as stated by our reference paper, we could achieve this by using a **soft-encoding** approach implemented with a *k-nearest neighbors* algorithm (with $k = 5$) applied in the output space and weighted with a *Gaussian kernel*. Using this function we can compute the ground truth Z and apply the loss on it.

3.4.3 Pixel Loss

We integrate a pixel loss component to achieve more vibrant and vivid colors in the distribution predicted by the generator. While we initially attempted to apply the loss only to the AB channels, the results were poor. Therefore, we opted for its standard implementation, using Mean Squared Error (MSE) over the entire predicted and ground truth LAB image.

$$PixelLoss = \frac{1}{n} \sum_{i=1}^n ||Y_i - \hat{Y}_i||^2$$

3.4.4 Generative Adversarial Loss

The Adversarial Loss for the generator is a classic **Binary Cross-Entropy loss** applied on a ground truth that is labeled as 1. It is formulated in a way that the Generator trains to create samples that can fool the discriminator, so the objective function is:

$$\max_g V(D, G) = E_{p_g(z)} [\log(D(G(Z)))]$$

² This would allow the generator to learn even if the discriminator is performing well.

3.4.5 Discriminative Adversarial Loss

The Adversarial Loss for the discriminator is instead composed of two elements, one that evaluates the prediction performed on the colored image, and another on the generated image. Mathematically, it can be described as:

$$\max_g V(D, G) = E_{p_d(x)} [\log(D(p_d(x)))] + E_{p_g(z)} [\log(1 - D(G(p_g(z))))]$$

where $p_d(x)$ refers to a colored image from the dataset. To

²Function that is maximized when the discriminator predicts 1 (real) over a generated sample

formulate it differently, we calculate a **BCE function** between the prediction on the colored drawing and 1, and another between the prediction of the generated colored image and 0. The total loss is the sum of both results.

3.5. Generator Loss

Summarizing all the functions described so far, the final version of our generator loss is: $Loss_g = \lambda_{adv} L_{adv} + \lambda_{MCE} L_{MCE} + \lambda_{pixel} L_{pixel}$

. To enhance the flexibility of our model, given that we employ various types of losses targeting different features and aspects of the computation, it could be beneficial to introduce weighting factors for the loss values. This would allow us to control the influence of each loss component during training, enabling better balance and offering more opportunities for fine-tuning the model.

3.6. Model Parameters

It is worth discussing how the loss in the first discriminator is managed, as our initial training tests revealed that its performance consistently outpaced that of the generator. This imbalance made it difficult for the generator to "keep up" with the discriminator's ability, hindering effective adversarial learning.. This is a known problem in GANs training, so we tried to resolve it in a different way, as assigning a lower learning rate was not enough. We added an additional hyperparameter, which is a threshold that defines the minimum value of the loss for the discriminator in order to perform the backward pass: if the loss is too low for a particular batch, the discriminator weights are not updated. We applied this technique only to the first discriminator, as the second one remained fairly balanced with the generator's performance.

We also tried different configurations of parameters and tested a series of combinations, as anticipated in *section 2.2*, by following the standard empiric scales for convolutional networks, in particular:

- For the *Zhang-only* network optimizer, we experimented with different learning-rate combinations between 0,0001 and 0,01, varying each time of a logarithmic step. However, the differences were negligible as the network, as we will see, failed to successfully colorize the images.
- For the *Adv Base* network optimizer, since the generator has difficulties in keeping up with the discriminator, we kept the discriminator's learning rate one order of magnitude lower compared to the one of the generator, throughout all test. Another characteristic is that this model seems to be more unstable (with respect to the PatchGAN), so we tried different combinations between 0,00002 and 0,002 (for both generator and discriminator optimizers) and found the best balance with

a very low learning rate value, as listed later. Also, to stabilize the convergence and allowing the optimizer to adapt better to quickly changing gradients, we lowered the *Beta_1* (Momentum) value of the optimizer. We tried to vary the *Discriminator Loss Threshold* between 0.6 and 1.2, finally choosing 1.0 for the final version.

- The same type of analysis and tests were done with the *PatchGAN* discriminator, but in this case we observed that the performance was fairly balanced between generator and discriminator, so after testing in the same intervals of values as the previous network, we decided to assign the same learning rate to both optimizers. The same *momentum* value was chosen, but we assigned a lower *Beta_2* value (0,9), to make the convergence quicker (knowing that this model is much more stable than the *Adv Base* one)

For both final versions of the discriminators, we decided to keep the *loss weights* at 1. We experimented with increasing the weight of the pixel loss—since the *adversarial* loss had the greatest influence, while the *Z loss* played a minor role—but this resulted in a spike in colorization inaccuracies and the appearance of unexpected colors. A possible reason why *PatchGAN* achieved more stable convergence could be the presence of dropout layers in its architecture. The first *Adv Base* model was trained with:

- **Generator Optimizer:** *Adam* optimizer, with a:
 - *Learning rate:* 0.0002
 - *Beta_1 Momentum:* 0.5
 - *Beta_2 RMSProp:* 0.999
- **Discriminator Optimizer:** the discriminator has also an *Adam* optimizer, with a:
 - *Learning rate:* 0.00002
 - *Beta_1 Momentum:* 0.5
 - *Beta_2 RMSProp:* 0.999
- **Discriminator Loss Threshold:** 1.0

The model with the *PatchGAN* discriminator was instead trained with:

- **Generator Optimizer:** *Adam* optimizer, with a:
 - *Learning rate:* 0.0002
 - *Beta_1 Momentum:* 0.5
 - *Beta_2 RMSProp:* 0.9
- **Discriminator Optimizer:** *Adam* optimizer, with a:
 - *Learning rate:* 0.0002

- *Beta_1 Momentum:* 0.5
- *Beta_2 RMSProp:* 0.9

Other significant values are listed here:

- **Dataset Mean and Standard Deviation:** values were computed over the entire One Piece Colored tables dataset and we obtained the following values, computing a single *mean* and *std* for both the A and B channels:
 - $L_{mean} = 73.07875061035156$
 - $L_{std} = 28.479511260986328$
 - $AB_{mean} = 3.4011828899383545$
 - $AB_{std} = 14.096550941467285$
- **Batch Size:** the reference value is 32 for every trained model. Initially we used batches of various, but lower, dimensions (2,4,8 images per batch). After having resolved an *out-of-memory* problem, related to loss gradients accumulations between batches, we chose 32, as a good balance between memory occupancy and training time. We could not go higher because of the limited computational resources.
- **Epochs:** we trained for 40 epochs in each generative model and only 25 epochs on the base (Zhang-Only) model.

4. Experiments

The main result we obtained during the first phase of experiments (on a subset of data) is the awareness that we had to change the normalization performed on the input images. Initially, we applied the same normalization proposed by the Zhang et al. paper [6], but this was a flawed choice because the images of our dataset are characterized by the presence of a white background and text (for most of them and mostly in the manga tables), as well as a less linear distribution of colors, as opposed to the one of real-world images. For this reason, we defined our own normalization values based on the test set distribution. This allowed the model to understand intrinsic characteristics of the drawings, like coloring in borders and, most importantly, assigning a more evenly and coherently distributed colors. One main observation that we can make about the first model's training is that, during our first tests, it was very unstable and not so robust, making choosing hyperparameter values to strike the best balance a fairly hard task. After training all the models described previously, specifically three, for each dataset (*safebooru* and *One Piece* tables), we can now evaluate the results.

4.1. Metrics used

We have already discussed how this type of task, especially applied to drawings, does not have a real solution, considering the opinionability of color choices and stylistic diversity. Following this principle, the correctness evaluation for our model would need to perform a statistical study over a sample of humans that try to distinguish between real- and fake-colored images. Of course, this is not feasible for us, so instead we used two quantitative metrics to evaluate the model(s):

- **Peak Signal-to-Noise Ratio (PSNR):** applied over a couple of tensors, allows to estimate the quantity of noise present in a *target* with respect to the ground truth. It's a valid measure because spatially, the generated and ground truth images are identical, so the PSNR computed would refer only to the colorization component.
- **Structural Similarity Index Measure (SSIM):** it's a well known metrics used to evaluate the perceived quality of images. It can be used to measure the similarity between two images, thereby assessing *image quality* in relation to a reference image.

Using quantitative metrics is therefore an acceptable choice in our study, mainly because our model is trained to work on a single manga, meaning that it should not only colorize the image in a plausible way, but also respect color patterns and coherence of the reference work (at least for the main characters). Hence, confronting the generator results with ground truth can give a good insight on the overall accuracy of the model (even if, in this case, we cannot talk of *accuracy* as a whole).

We have to remember that, knowing how colors are distributed in AB channels, even with a sepia tone and poorly colored images, the result of PSNR and SSIM is still acceptable.

4.2. Zhang-Only Network

The main issue with using only the Zhang network, as previously discussed, is that the colorization loss is not supported by **class rebalancing**. Class rebalancing is one of the key components that contributed to the strong results presented in the original paper. As a result, the obtained output is unacceptable, failing to effectively colorize images. This is also linked to how color probabilities for drawings are distributed in **H space** and the fact that, without rebalancing, the dimensionality remains very limited. In other words, while the loss value is quite low, the quality of the is similarly poor.

Furthermore, the convergence observed in the graph (Fig. 5,6) is misleading, as the loss decreases at an extremely slow rate and ultimately fails to converge.

Model	Test Loss (Gen)	Test Loss (Disc)	PSNR (dB)	SSIM
Zhang (SB)	0.0149	—	≈ 23.3	≈ 0.472
Zhang (OPT)	0.0133	—	≈ 24.5	≈ 0.453
Adv Base Discriminator (SB)	1.217	5.2787	≈ -23.2	≈ 0.540
Adv Base Discriminator (OPT)	1.520	3.0989	≈ -20.5	≈ 0.694
PatchGAN Discriminator (SB)	0.622	2.609	≈ -22.9	≈ 0.592
PatchGAN Discriminator (OPT)	3.320	0.559	≈ -20.6	≈ 0.699

Table 1. Results. *OPT: One Piece Tables dataset; SB: Safebooru dataset*

4.3. Model with first discriminator

This model represents our first major milestone, as it successfully achieves image colorization. Evaluating the performance on the *One Piece Tables* dataset, the main observation we can make is that, compared to the ground truth colorization, it often produces *sepia-toned* and *desaturated* results. Additionally, in manga panels, the model tends to apply a uniform color grade across most parts of the image. This means that, when dealing with drawings representing scenes with a large and complex background, the same AB values are used for both the background and nearby characters, resulting in color contamination and noise in certain areas.

In some cases, the characters' colors blend with the background, and text vignettes may exhibit a high level of noise. However, the model performs well when handling the main characters, showing confidence in assigning the correct color patterns. It also generally succeeds in coloring within the edges, especially when the illustration is simple and the characters are properly positioned. On the other hand, the results obtained with the *Safebooru* dataset are highly approximate, and no visually successful cases were found in the colorized test set. The main reason for this is that all drawings are desaturated, with lot of noise and color dials (as we can see in Fig. 2).

Fig 7 and 8 plots training and validation loss for this model.

4.4. Model with patch gan discriminator

The main difference from the first model, when using *One Piece Tables* dataset, is that this architecture handles complex scenes and small-sized characters much better, despite having similar issues with complex backgrounds. In fact, the model tends to be biased toward green when colorizing backgrounds, which can also affect characters, particularly when they appear in specific positions or in figures that occur infrequently in the manga. Unfortunately, also with this model we had no good results on the *Safebooru* dataset.

Fig. 9 and 10 plots training and validation loss.

4.5. Results evaluation

The results obtained are obviously constrained by our computational resources. However, the low grade of convergence is not a primary issue because we know that in GANs, training loss of both discriminator and generator

are unstable because of their mutual influence. Hence, we would likely observe a higher degree of convergence with 500 or 1000 training epochs, especially when using a larger dataset. Also, it is difficult for this task to define an objective loss value, considering that multiple and different types of loss are combined in the generator.

We can also summarize and analyze the reasons behind the failure of the models trained on the *Safebooru* dataset: looking at the results obtained with the other dataset, we can suppose that the model failed to learn recurrent spatial features associated with characters, objects and environments, a task that is much easier when applied on the manga tables, which are characterized by a lot of repeated shapes and characters are drawn always the same way. On the other hand, as discussed in *section 2*, *Safebooru* has highly sparse data (in terms of colors) and even the same character is drawn in a wide variety of styles, shapes and colors.

In conclusion, the more approximate results obtained by the *Adv Base* model can be related to a mode collapse during the training, given that, in the test set, the generator loss is lower than the discriminator's, that is instead very high. Also, the fact that the discriminator loss for the PatchGAN is near $1/2$, demonstrates the quality of the obtained result, as it is a value close to the *Nash equilibrium* for GANs training.

4.6. Metrics Evaluation

An interesting, though limited, perspective on the models' performance is provided by the pair of quantitative metrics we used: PSNR and SSIM. As we can see from the results, those values are only indicative and only partly reflect the visual results in Table 2. The SSIM is higher and near to 0.7 in the *PatchGAN* with *One Piece Tables* dataset, correctly describing the visual quality of the resulted image with respect to the original and the trend of the results. On the other hand, the PSNR is even negative for all the adversarial models, indicating an higher level of *noise* than of the perceived signal. This is understandable, as there are dozens of failure cases in the colorized test set and thus, where drastic color shifts or inconsistencies (or both) contribute to the predominance of *noise* over the original signal. This interpretation is further supported by the results of the *Zhang* network, which, despite failing to colorize the images, achieves a PSNR above $20dB$, an acceptable value in general but not in the context of our study.

4.7. Comparing with other works

Before trying to compare our model(s) with the work of others, we have to remember that, as already stated previously, a "real" end-to-end solution of drawing colorization is not available and other studies have tried to cover the task by adding additional information to the gray-scale input, such as:

- The flat colored image [4]: in this work, a generative model takes as input the shading and flat colored version of the same drawing so the network can fuse the two information and obtain the final colored image
- Reference image from which the specific character's color pattern is taken [1]: this is one of the most powerful solution in the state of the art, based on colorizing a drawing of a particular character by using a visual (colored) reference of the same figure. The main characteristic (and difference) of this network is that it uses a diffusion model.
- Manual color hints palette [5]: it's a method based on a two stage elaboration, in which the network produces a "first guess" for the colorization and then the result is corrected in a refinement stage. In both stages a series of hints have to be over-imposed to the image, first drafts and then a refined version.

So, the issue is not only related to how forward propagation is executed but, more importantly, to the dataset and the additional information required to effectively use and train the model. For all these reasons, we cannot make a fair quantitative or qualitative comparison between our model and previous works.

5. Conclusions

In this work, we explored the automatic colorization of manga using neural networks, achieving visually acceptable results despite the inherent challenges posed by the task. One of the main issues we encountered was the lack of diversity and the non-homogeneity of the dataset (mainly in *Safebooru*), which affected the generalization capability of the model. However, within the constraints of our experimental setup, the generated colorizations exceeded the initial expectations, demonstrating the model's ability to learn and apply meaningful color distributions to the input images, even with poor results on metrics. This stresses the concept that this type of task cannot be quantitatively evaluated without a survey on human perception. A key factor that contributed to the success of our approach is the focus on a single manga. This specialization allowed the model to effectively learn the recurring visual patterns, shapes, and color relationships specific to this work, leading to more coherent and contextually appropriate colorizations. However, this also represents a limitation: the model is highly domain-specific and cannot generalize well to other manga or drawing styles without significant retraining on a more diverse dataset (and this is clear when training on the *Safebooru* dataset).

Future developments in automatic *drawing* colorization could benefit from integrating additional input data, as suggested in other works. For instance, incorporating semantic

segmentation and combining it with classification, as happens in other works focused on real images colorization, can be a possible evolution of our architecture. Such improvements would help mitigate the ambiguities inherent in grayscale-to-color mapping and allow for more precise and "realistic" results, given that there is no *real* and *correct* colorization in drawings.

Appendix

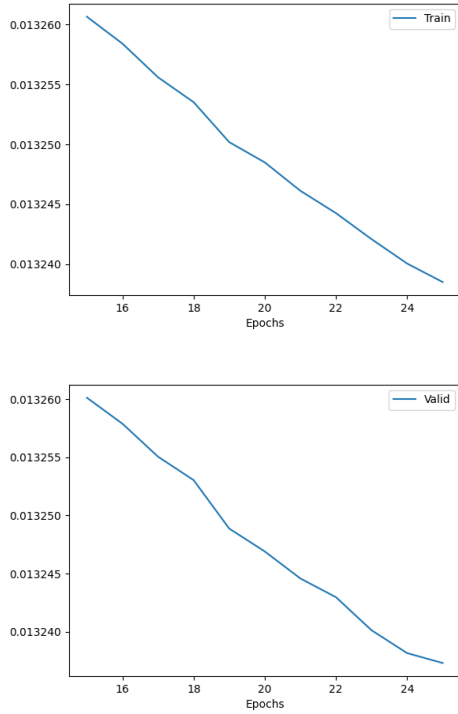


Figure 5. Train and Valid loss for Zhang model (One Piece Tables d.)

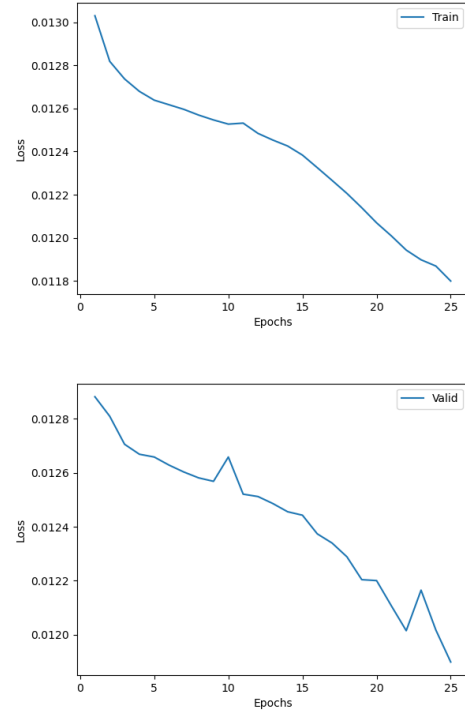


Figure 6. Train and Valid loss for Zhang model (Safebooru d.)

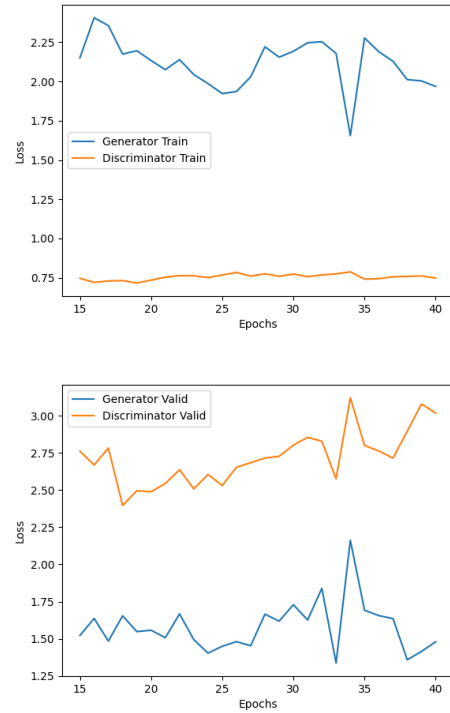


Figure 7. Train and Valid loss for "Adv Base" (One Piece Tables d.)

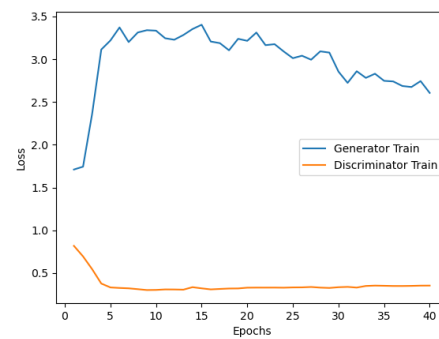
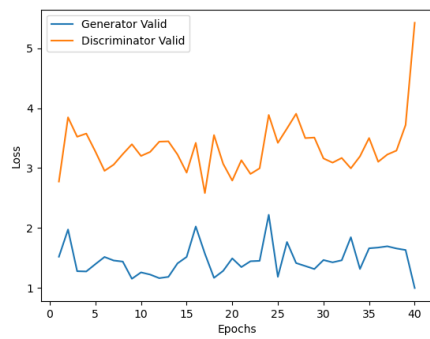
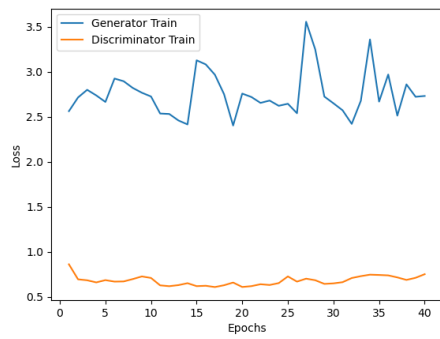


Figure 8. Train and Valid loss for "Adv Base" (Safebooru d.)

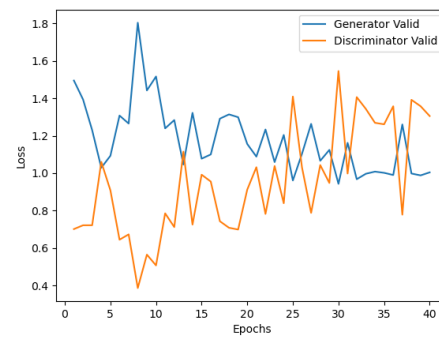
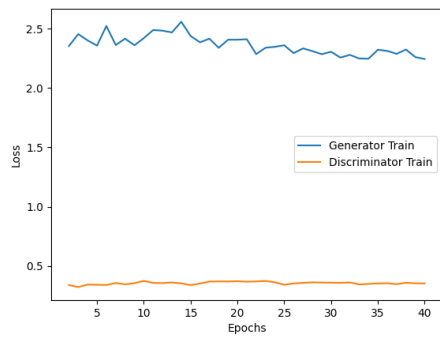


Figure 10. Train and Valid loss for PatchGAN model (Safebooru d.)

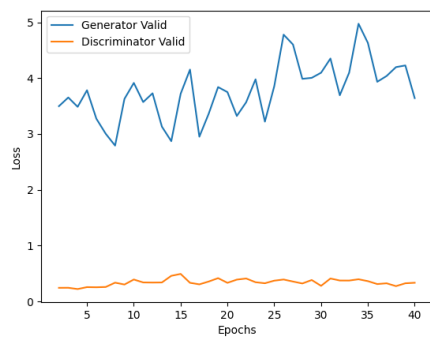


Figure 9. Train and Valid loss for PatchGAN model (One Piece Tables d.)

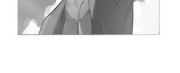
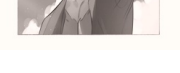
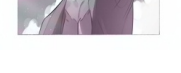
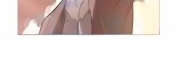
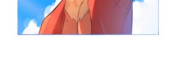
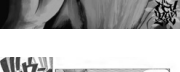
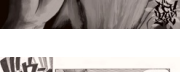
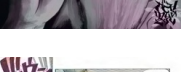
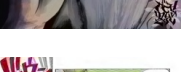
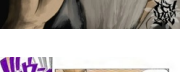
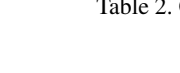
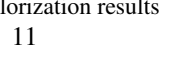
	Input	Zhang Only	Adv Base Model	PatchGAN Model	Ground Truth
Success cases					
					
					
					
Failure cases					
					
					
					
Failure cases					
					
					
					
Failure cases					
					
					
					
Failure cases					
					
					
					
Failure cases					
					
					
					

Table 2. Colorization results

References

- [1] CAO, Y., MENG, X., MOK, P. Y., LIU, X., LEE, T.-Y., AND LI, P. Animediffusion: Anime face line drawing colorization via diffusion models, 2023.
- [2] ISOLA, P., ZHU, J.-Y., ZHOU, T., AND EFROS, A. A. Image-to-image translation with conditional adversarial networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (July 2017).
- [3] RADFORD, A. Unsupervised representation learning with deep convolutional generative adversarial networks. *arXiv preprint arXiv:1511.06434* (2015).
- [4] SHIMIZU, Y., FURUTA, R., OUYANG, D., TANIGUCHI, Y., HINAMI, R., AND ISHIWATARI, S. Painting style-aware manga colorization based on generative adversarial networks. 1739–1743.
- [5] ZHANG, L., LI, C., WONG, T.-T., JI, Y., AND LIU, C. Two-stage sketch colorization. *ACM Transactions on Graphics (SIGGRAPH Asia 2018 issue)* 37, 6 (November 2018), 261:1–261:14.
- [6] ZHANG, R., ISOLA, P., AND EFROS, A. A. Colorful image colorization. In *Computer Vision – ECCV 2016* (Cham, 2016), B. Leibe, J. Matas, N. Sebe, and M. Welling, Eds., Springer International Publishing, pp. 649–666.